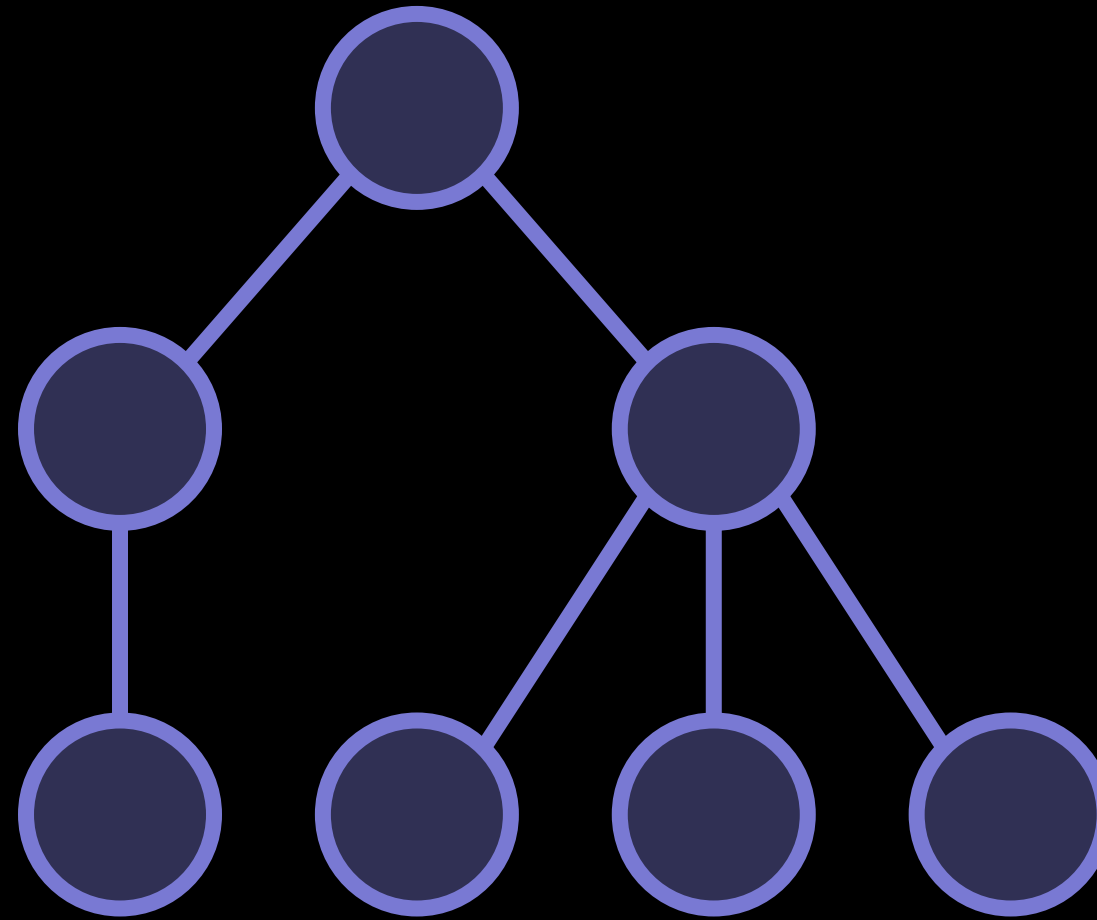


2. DOM과 이벤트 I



DOM이란?



문서 객체 모델 (Document Object Model)

구조화된 문서(XML, HTML)를 객체 형태로 표현하는 방식



DOM의 트리 구조

```
<!DOCTYPE html>
<html>
<head>
  <title>자바스크립트 기초</title>
</head>
<body>
  <article>
    <header>header</header>
    <section>
      <header>header 1</header>
      section 1
    </section>
  </article>
</body>
</html>
```



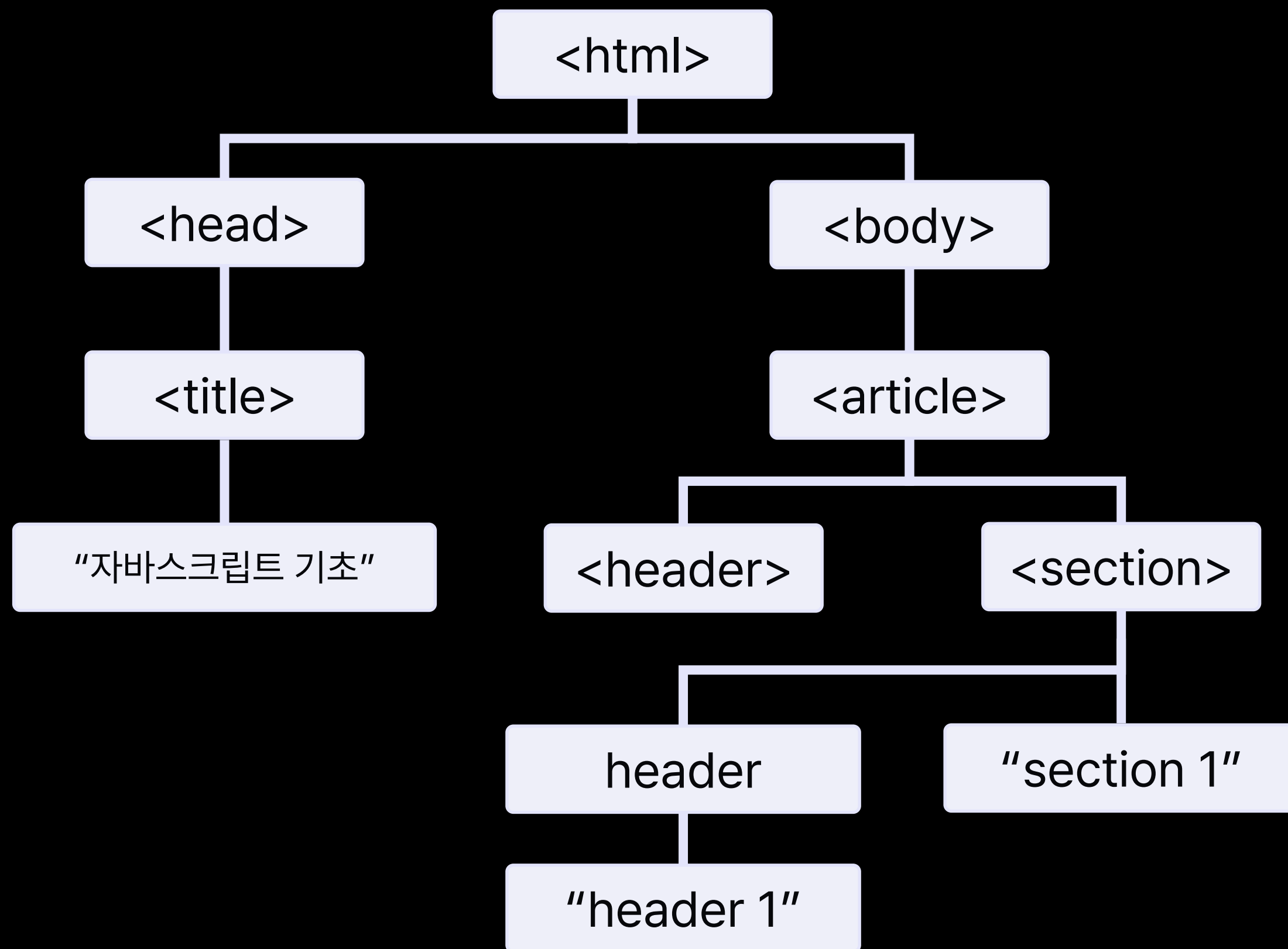


DOM의 트리 구조





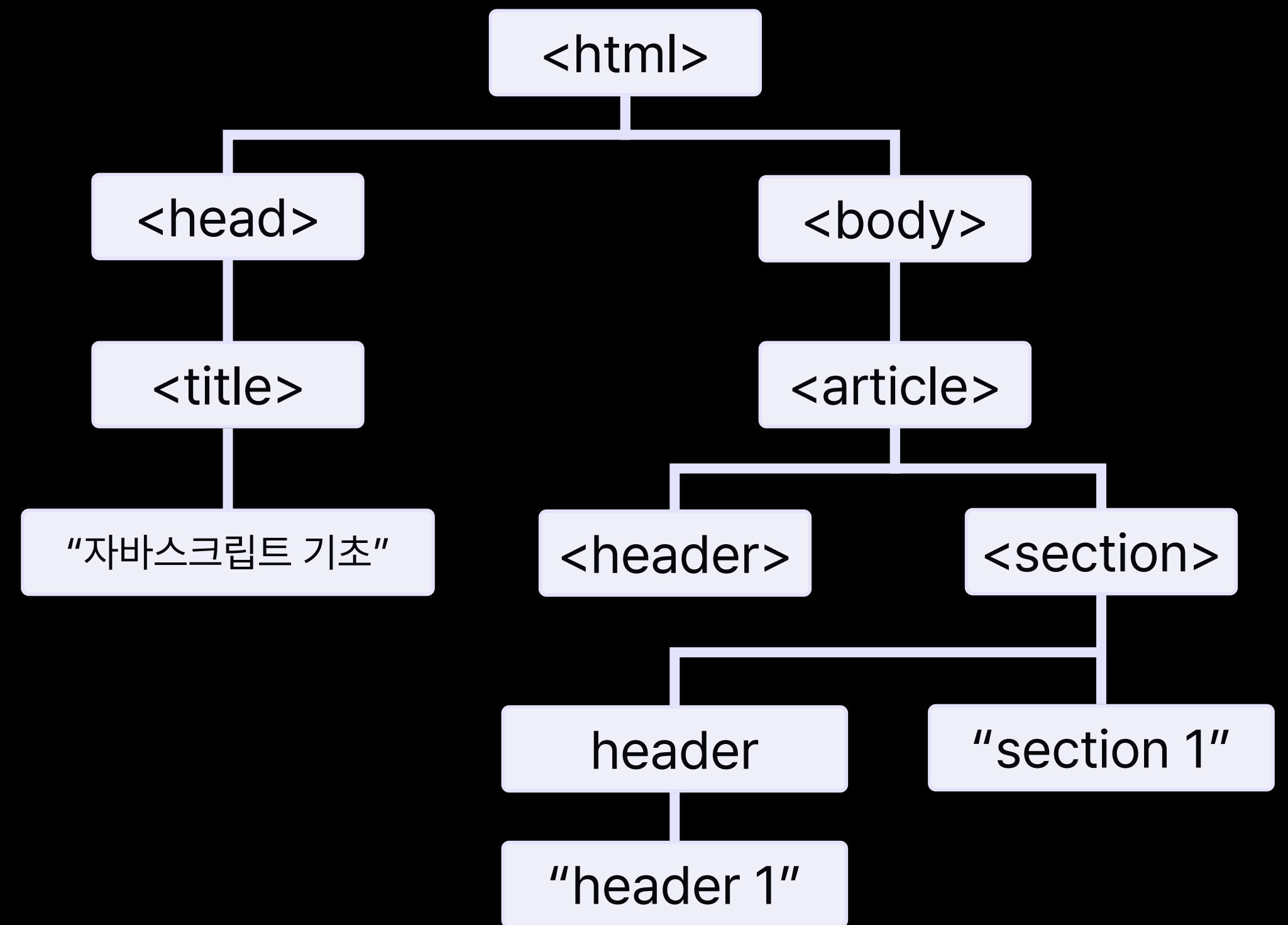
DOM의 트리 구조





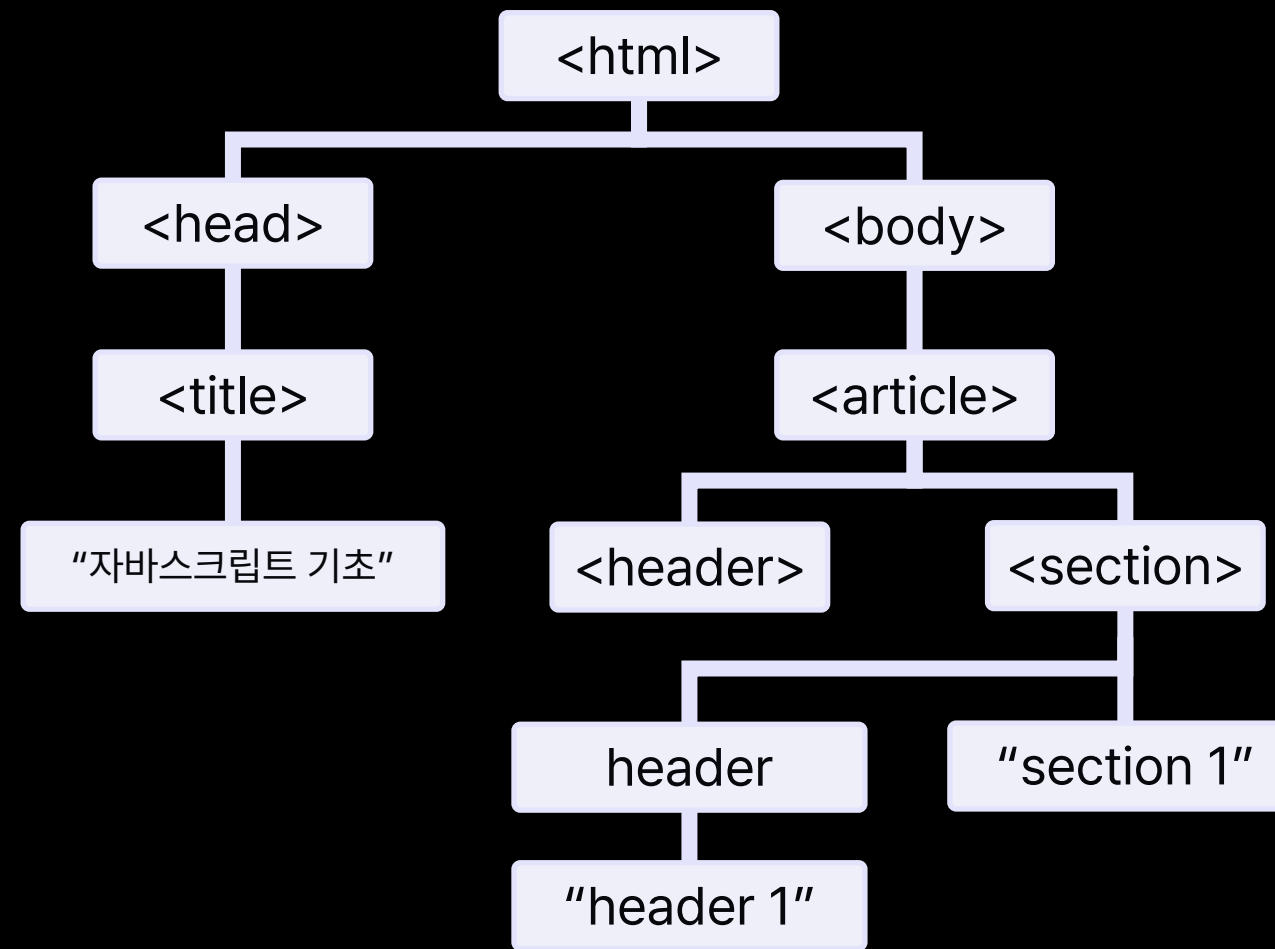
DOM의 트리 구조

```
<!DOCTYPE html>
<html>
<head>
  <title>자바스크립트 기초</title>
</head>
<body>
  <article>
    <header>header</header>
    <section>
      <header>header 1</header>
      section 1
    </section>
  </article>
</body>
</html>
```



DOM은 자바스크립트 엔진이 HTML 코드를 해석한 결과이다.

각각의 태그가 자바스크립트의 **node 객체**로 변환된다.



```
> document.querySelector(".css-dnmq8v")
< <p class="MuiTypography-root MuiTypography-h3 css-dnmq8v">도입 고객사 수</p>
```

노드: HTML DOM에서 정보를 저장하는 계층적 단위

노드 트리: 노드들의 집합으로, 노드 간의 관계를 트리 구조로 나타낸 것



document 객체

```
> document
< ▼ #document (https://elice.io/ko)
  <!DOCTYPE html>
  <html lang="ko" style="--event-banner-display: block;">
    ▶ <head> (...) </head>
    ▶ <body> (...) </body>
  </html>
```

document 객체는 웹 페이지를 의미한다.

JavaScript에서 DOM에 접근하고자 할 때에는

document 객체를 사용해야 한다.


```
> document.querySelector(".css-dnmq8v").  
< is {tag: 5, key: null, elementType: 'p'  
accessKey  
addEventListener  
after  
align  
animate  
append  
appendChild  
ariaAtomic  
ariaAutoComplete  
ariaBrailleLabel  
ariaBrailleRoleDescription
```

node 객체에는 해당 요소를 조작할 수 있는
수많은 프로퍼티와 메서드가 존재한다.



document, node 객체

HTML 요소의 선택과 조작

HTML 요소의 생성 및 추가

HTML 이벤트 핸들러 추가

document와 node 객체는 위와 같이

HTML 요소와 관련된 작업을 도와주는 다양한 메서드를 제공한다.



HTML 요소의 선택

메서드 사용 예시	설명
<code>document.getElementById("box")</code>	해당 <u>아이디</u> 의 요소를 선택
<code>document.getElementsByClassName("list-item")</code>	해당 클래스에 속한 요소를 모두 선택
<code>document.getElementsByName("email")</code>	해당 <u>name</u> 속성값을 가지는 요소를 모두 선택
<code>document.querySelectorAll("ul > li")</code>	해당 <u>CSS 선택자</u> 로 선택되는 요소를 모두 선택
<code>document.querySelector("header > #title")</code>	해당 <u>CSS 선택자</u> 로 선택되는 요소 중 첫 번째 요소 선택



HTML 요소의 선택

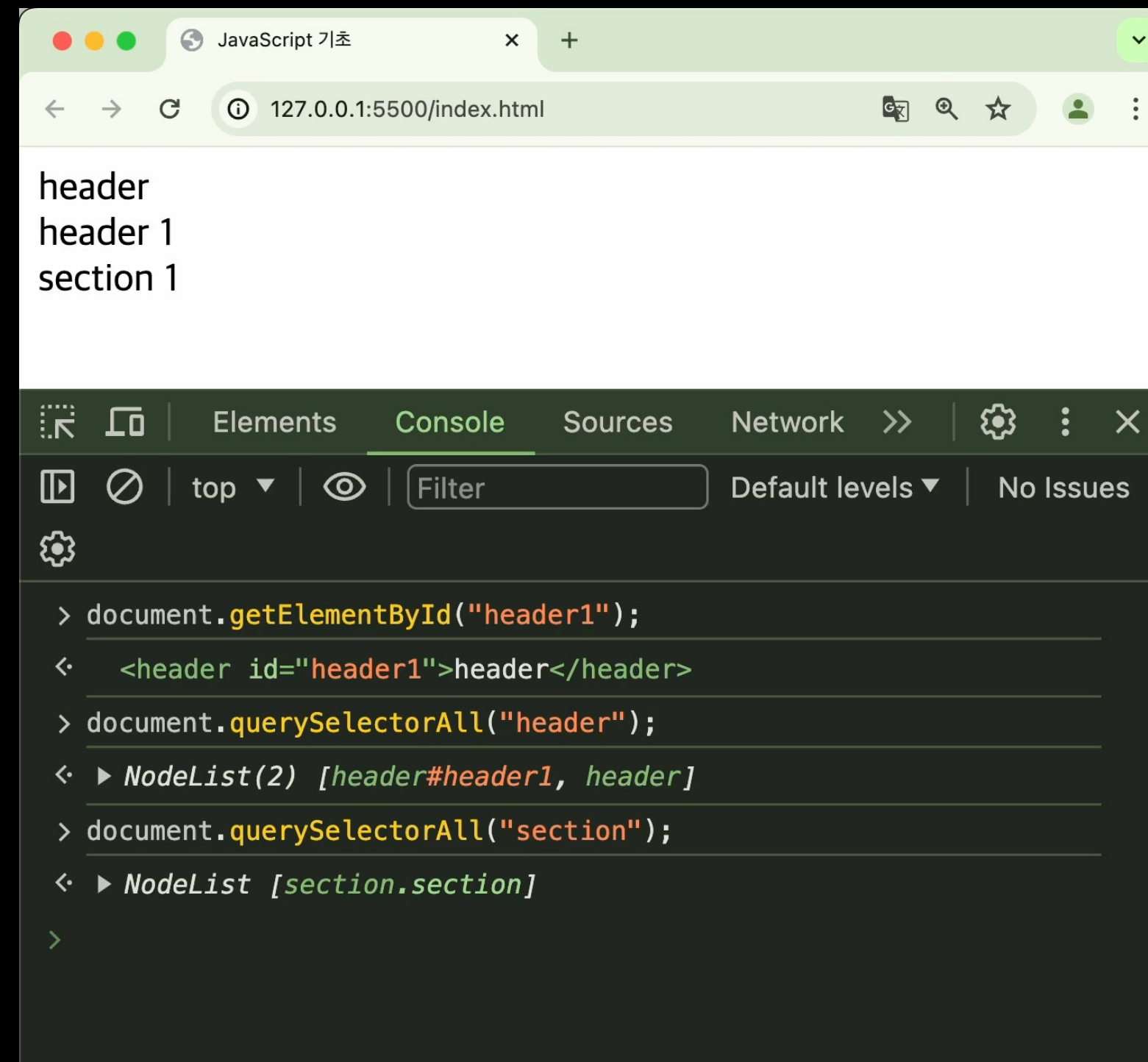
```
// HTML <li> 요소를 선택
var selectedItem = document.getElementsByTagName("li");

// 아이디가 "id"인 요소를 선택
var selectedItem = document.getElementById("id");

// "odd" 클래스를 가진 모든 요소를 선택
var selectedItem = document.getElementsByClassName("odd");

// name 속성 값이 "first"인 모든 요소를 선택
var selectedItem = document.getElementsByName("first");
```


HTML 요소의 선택 - 하나 / 모두 선택



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5500/index.html". The page content displays "header", "header 1", and "section 1". The browser's developer tools are open, with the "Console" tab selected. The console shows the following JavaScript commands and their outputs:

```
> document.getElementById("header1");  
< <header id="header1">header</header>  
> document.querySelectorAll("header");  
< ▶ NodeList(2) [header#header1, header]  
> document.querySelectorAll("section");  
< ▶ NodeList [section.section]  
>
```

Below the browser window, the word "Play" is written in white, followed by a purple arrow pointing to the right.



메서드 사용 예시	설명
요소.style.스타일속성 = "200px"	요소의 스타일 조회 및 변경
요소.innerHTML = " hello "	요소 하위의 HTML 조회 및 변경
요소.innerText = "Hello Elice"	요소 하위의 텍스트 조회 및 변경
요소.getAttribute("placeholder") 요소.setAttribute("min", "20")	요소의 HTML 속성 조회 및 변경
요소.classList = "button button--red"	요소의 class 수정
...	

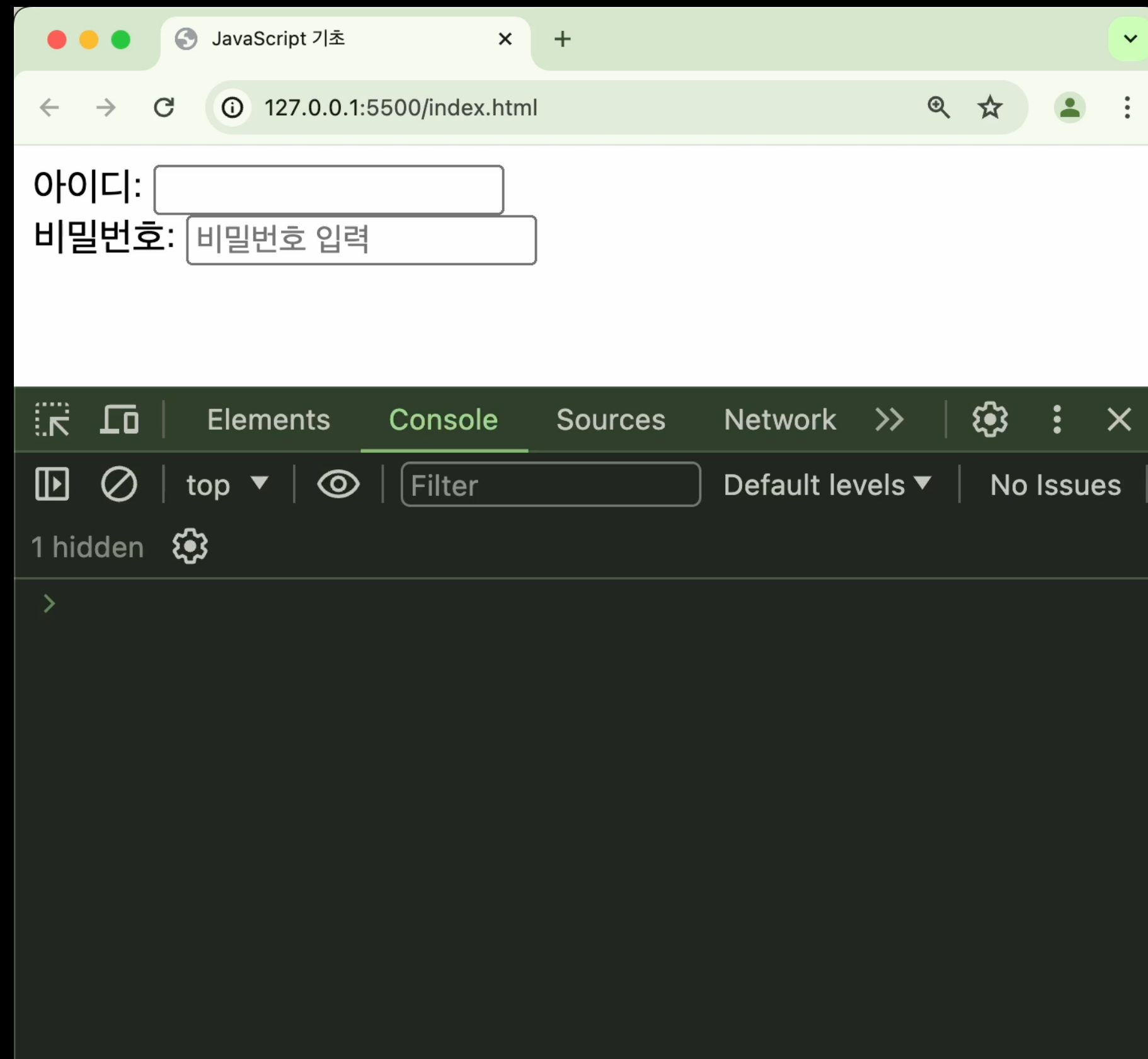


HTML 요소의 조작 – 스타일 변경

```
// 클래스가 "even"인 요소를 선택
var selectedItem = document.querySelector(".even");

// 선택된 요소의 텍스트 색상을 변경
selectedItem.style.color = "red";
```

HTML 요소의 조작 – 스타일 변경



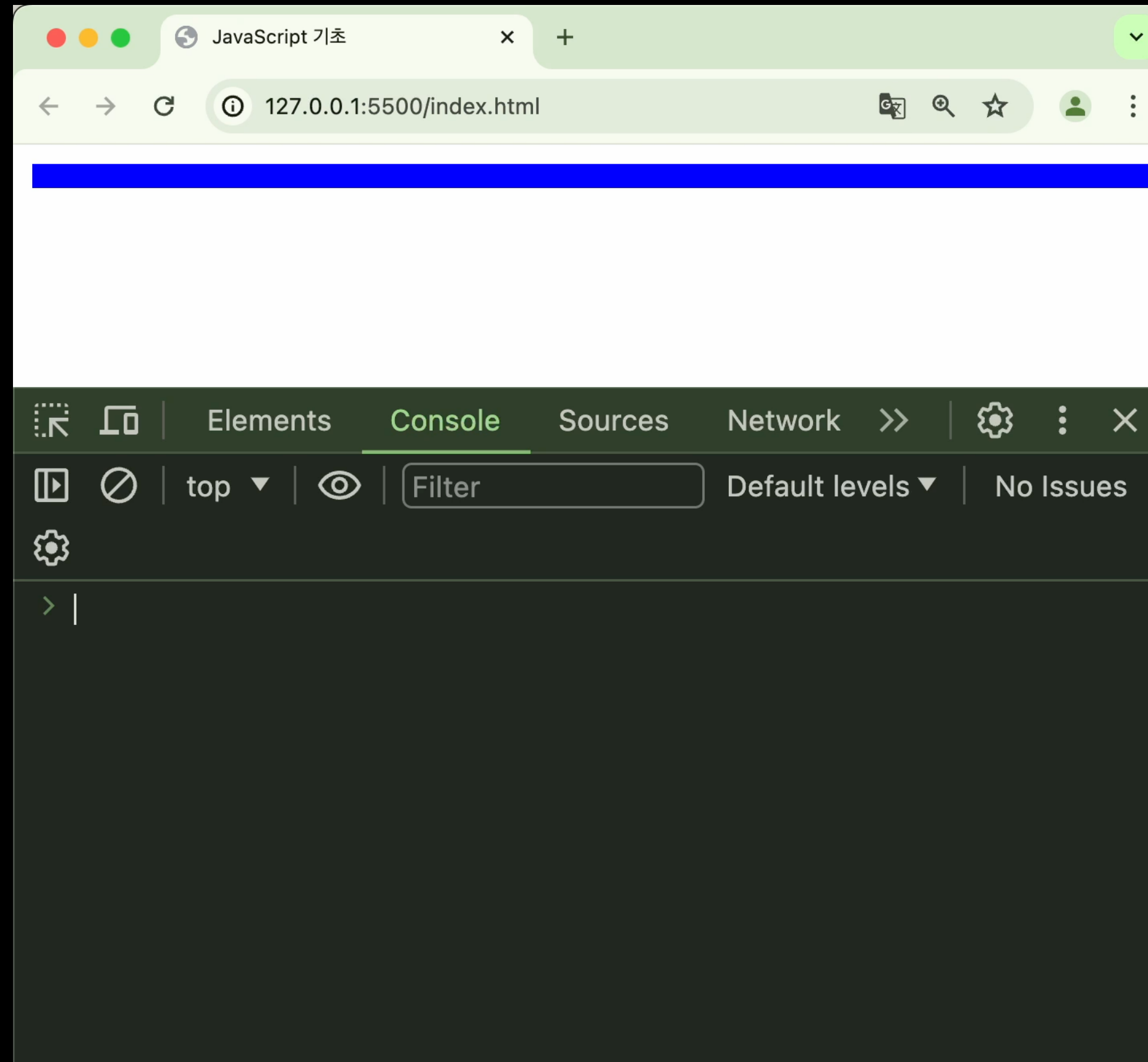
Play →



HTML 요소의 조작 – innerHTML

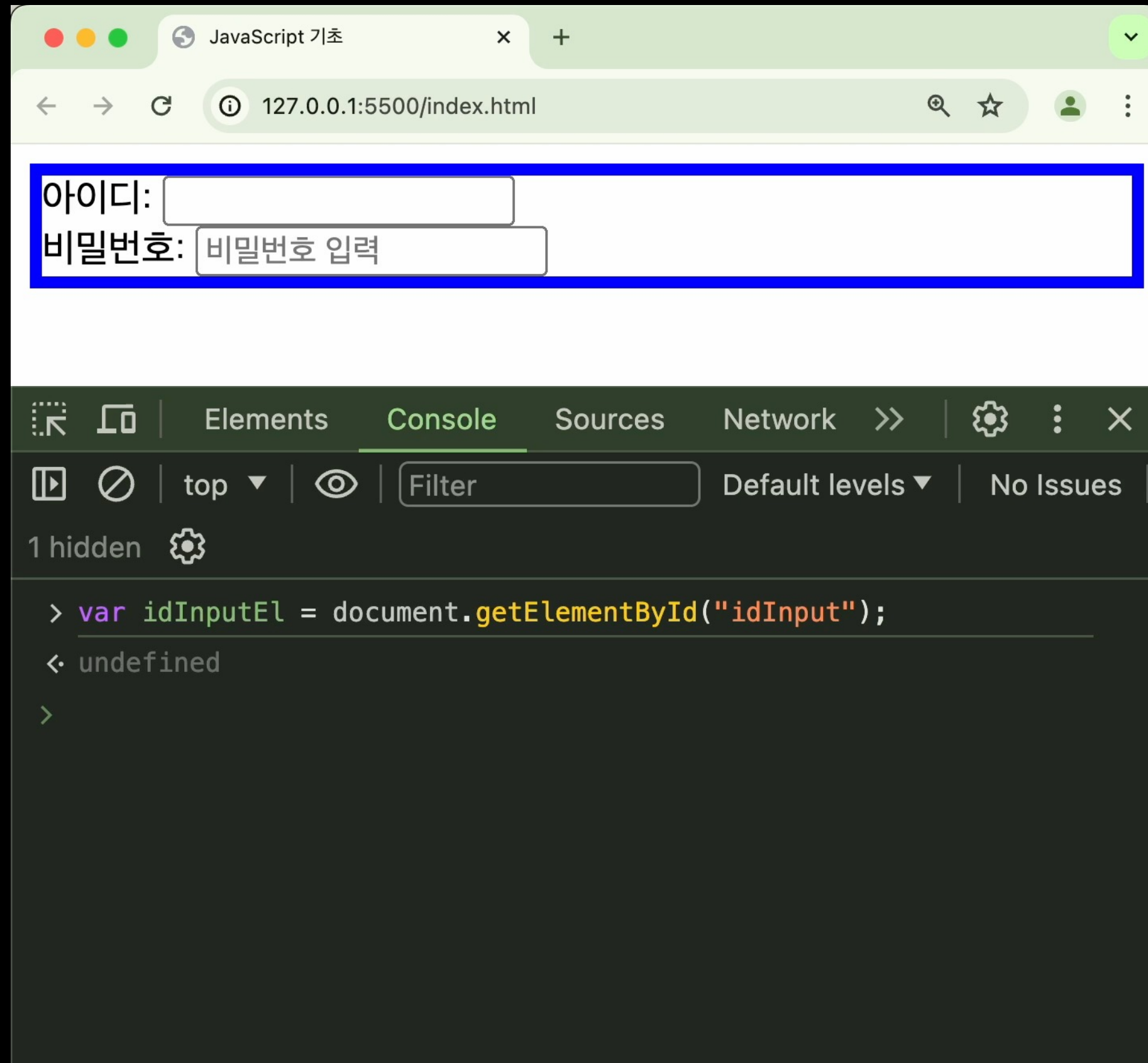
```
// 아이디가 "text"인 요소를 선택  
var str = document.getElementById("text");  
  
// 선택된 요소의 하위 HTML을 변경  
str.innerHTML = "<span>안녕하세요</span>";
```


HTML 요소의 조작 – innerHTML



Play →

HTML 요소의 조작 - 속성 조회 및 변경



The screenshot displays a web browser window with the title "JavaScript 기초" and the address bar showing "127.0.0.1:5500/index.html". The page content includes a form with two input fields: "아이디:" and "비밀번호:". The "비밀번호:" field contains the text "비밀번호 입력".

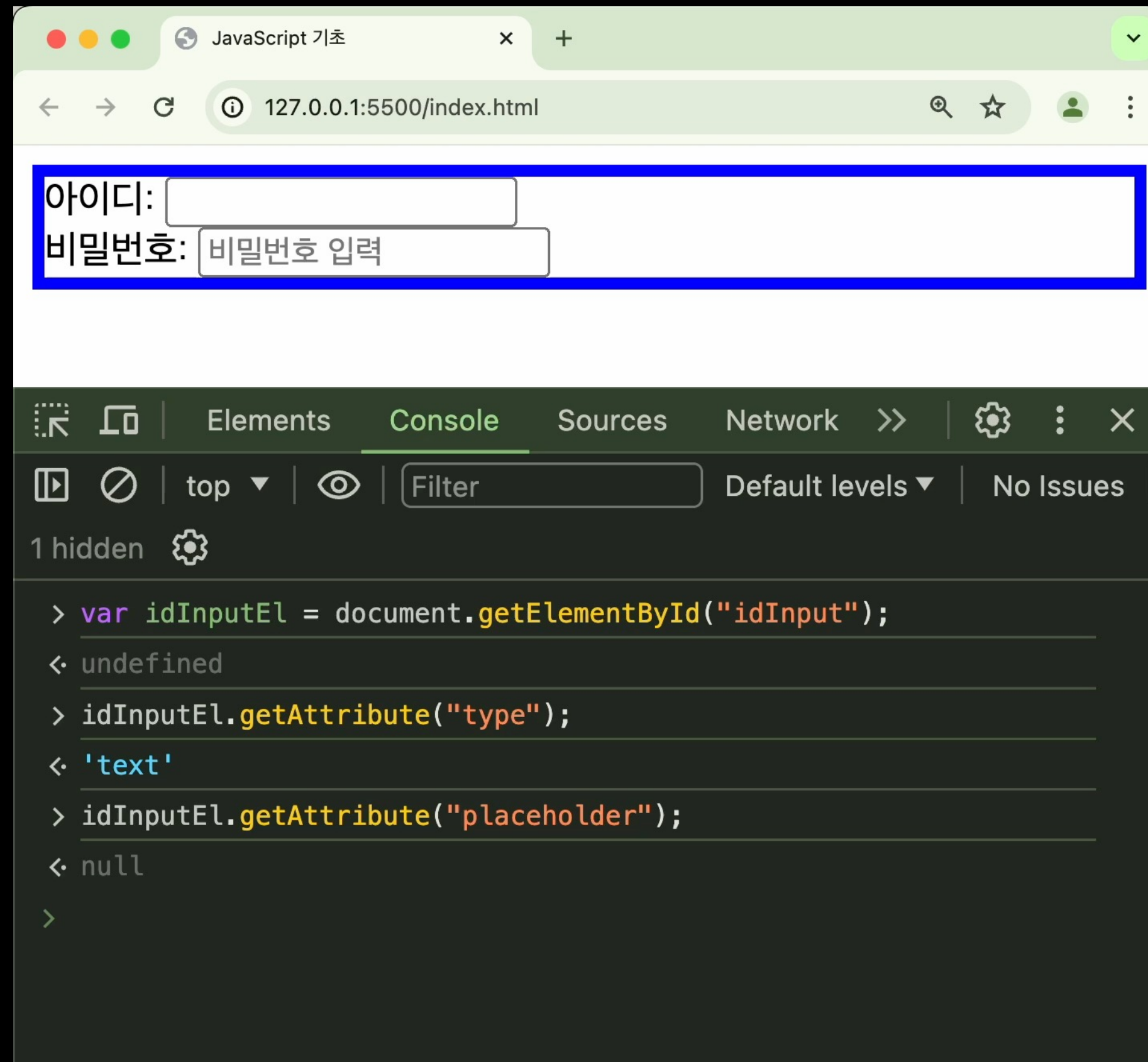
The developer console is open, showing the following JavaScript code and its output:

```
> var idInputEl = document.getElementById("idInput");  
< undefined  
>
```

The console indicates that the `getElementById` method returned `undefined`, suggesting that the element with the ID `idInput` was not found in the document.

Play →

HTML 요소의 조작 - 속성 조회 및 변경



아이디:

비밀번호:

Elements Console Sources Network >> | Settings | No Issues

1 hidden

```
> var idInputEl = document.getElementById("idInput");
< undefined
> idInputEl.getAttribute("type");
< 'text'
> idInputEl.getAttribute("placeholder");
< null
>
```

Play →

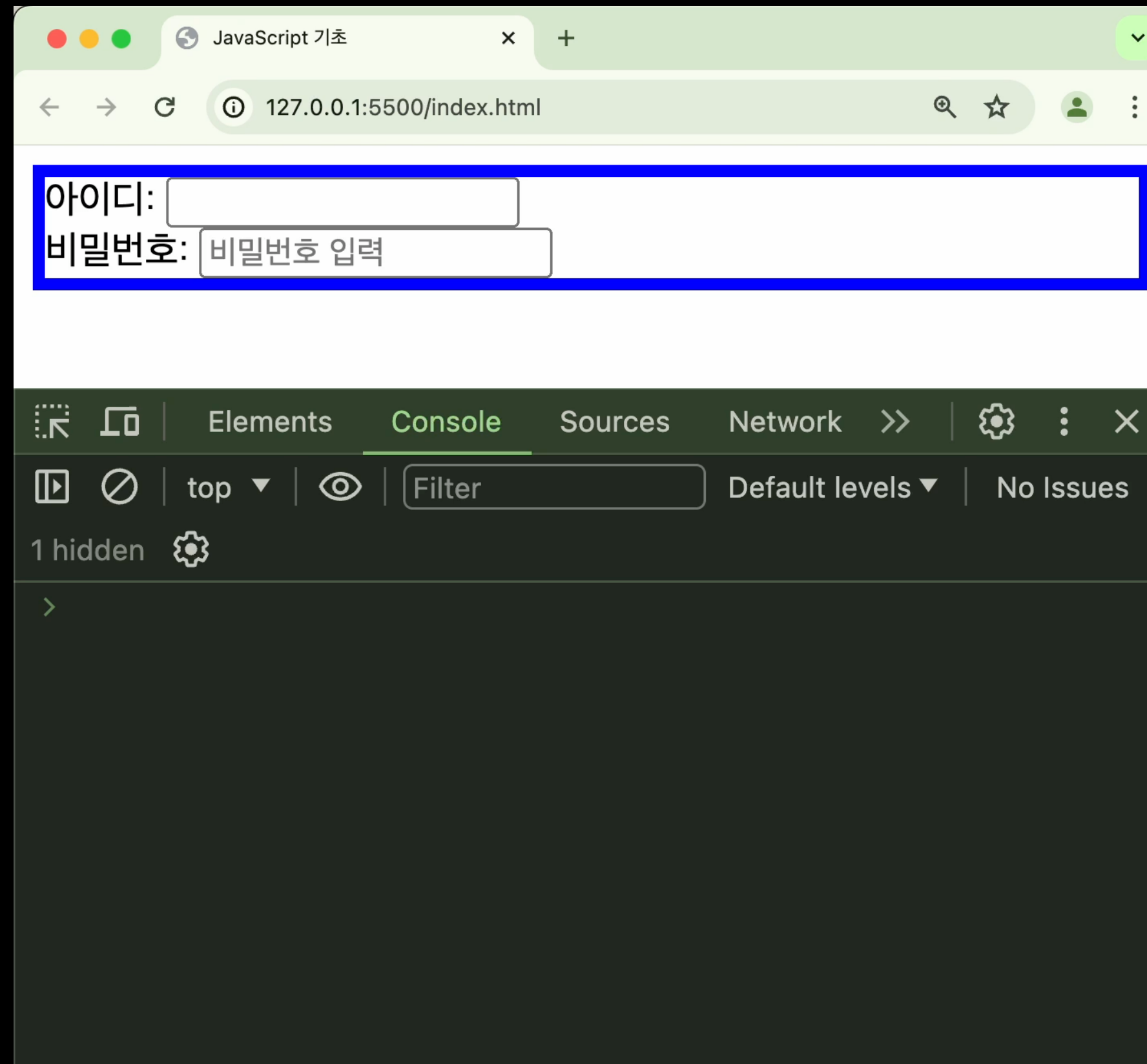


HTML 요소의 생성 및 추가

메서드 사용 예시	설명
<code>document.createElement("form")</code>	해당 태그의 HTML 요소를 생성
<code>document.write("Hello World")</code>	텍스트를 화면에 출력
<code>요소.appendChild(node)</code>	해당 요소 내부 가장 마지막에 node 추가



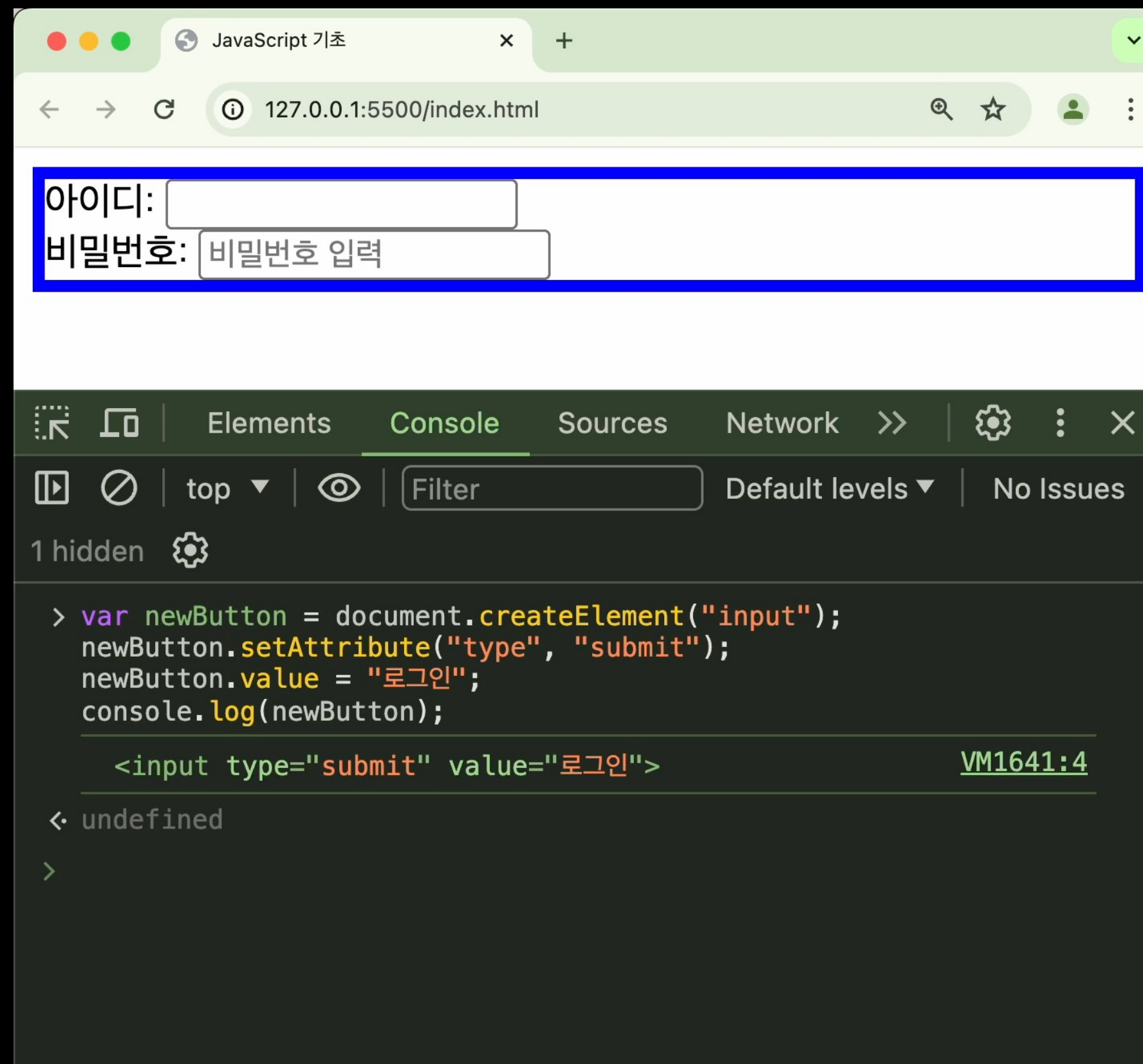
HTML 요소의 생성



Play →



HTML 요소의 추가



Play →

‘createElement’로 생성된 노드는 처음엔 자바스크립트 상에서만 존재한다.
화면에 보여지기 위해 ‘appendChild’ 등을 통해 DOM에 추가되어야 한다.

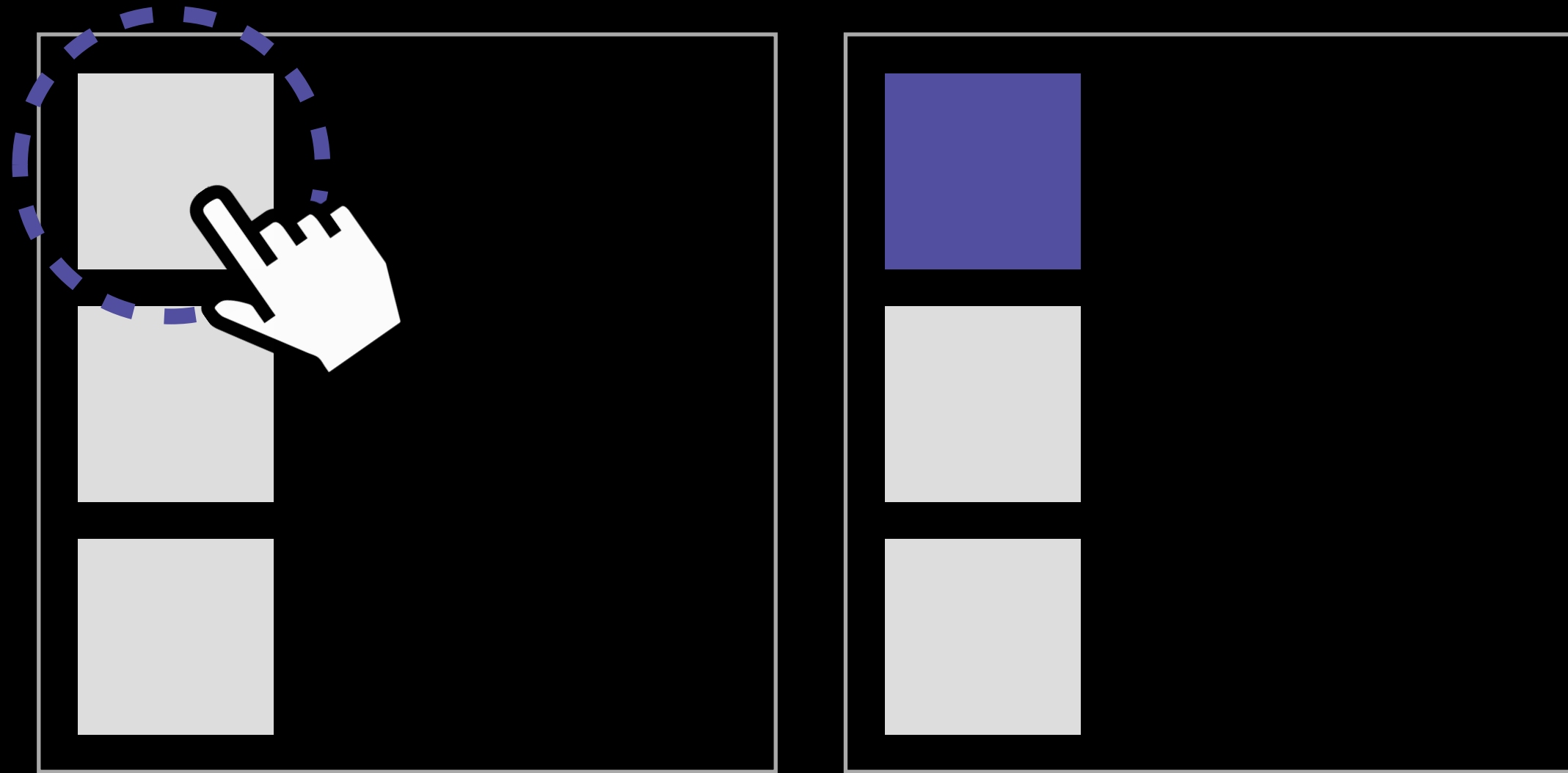


HTML 이벤트 핸들러 추가

사용 예시	설명
<pre><요소 onclick="handler()" /></pre>	요소의 여러 이벤트에 대해 이벤트 핸들러 연결
<pre>function handler() { ... }</pre>	
<pre>요소.onclick = function() { ... } 요소.onsubmit = function() { ... } ...</pre>	
<pre>요소.addEventListener("click", function() { ... })</pre>	



이벤트란?



웹 브라우저가 알려주는 HTML 요소에 대한 **사건의 발생**

자바스크립트는 발생한 이벤트에 반응하여 특정 동작을 수행할 수 있다.



이벤트 종류

마우스 이벤트

마우스 조작과
관련된 이벤트

키 이벤트

키보드 조작과
관련된 이벤트

폼 이벤트

form 태그 관련
동작에 대한 이벤트

이 외에도 로드 관련, 창 관련 이벤트 등 존재

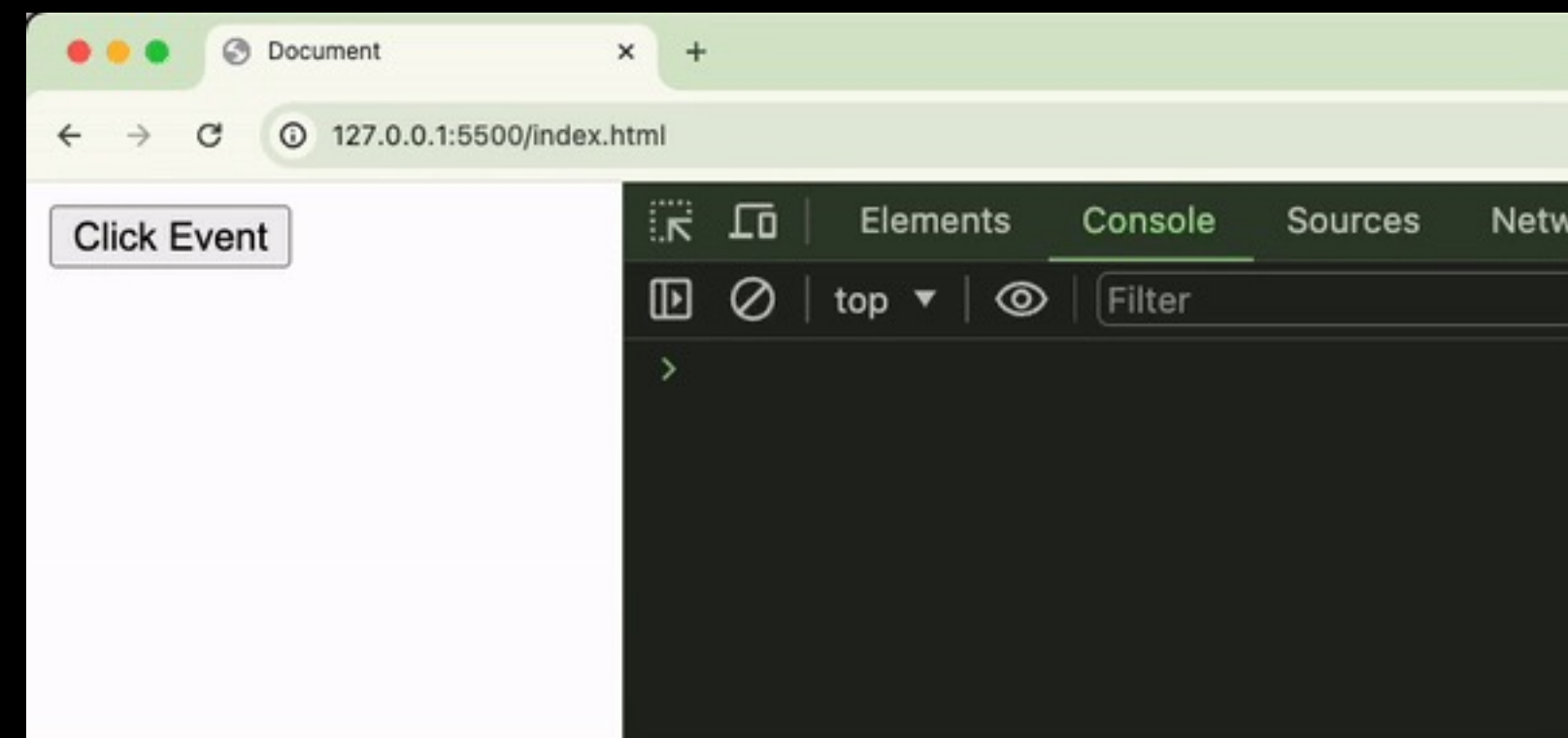
<https://developer.mozilla.org/en-US/docs/Web/Events>



이벤트 핸들러

```
var button = document.getElementById("btn");

button.addEventListener("click", function () {
    console.log("button clicked");
});
```



이벤트가 발생했을 때 처리를 담당하는 함수

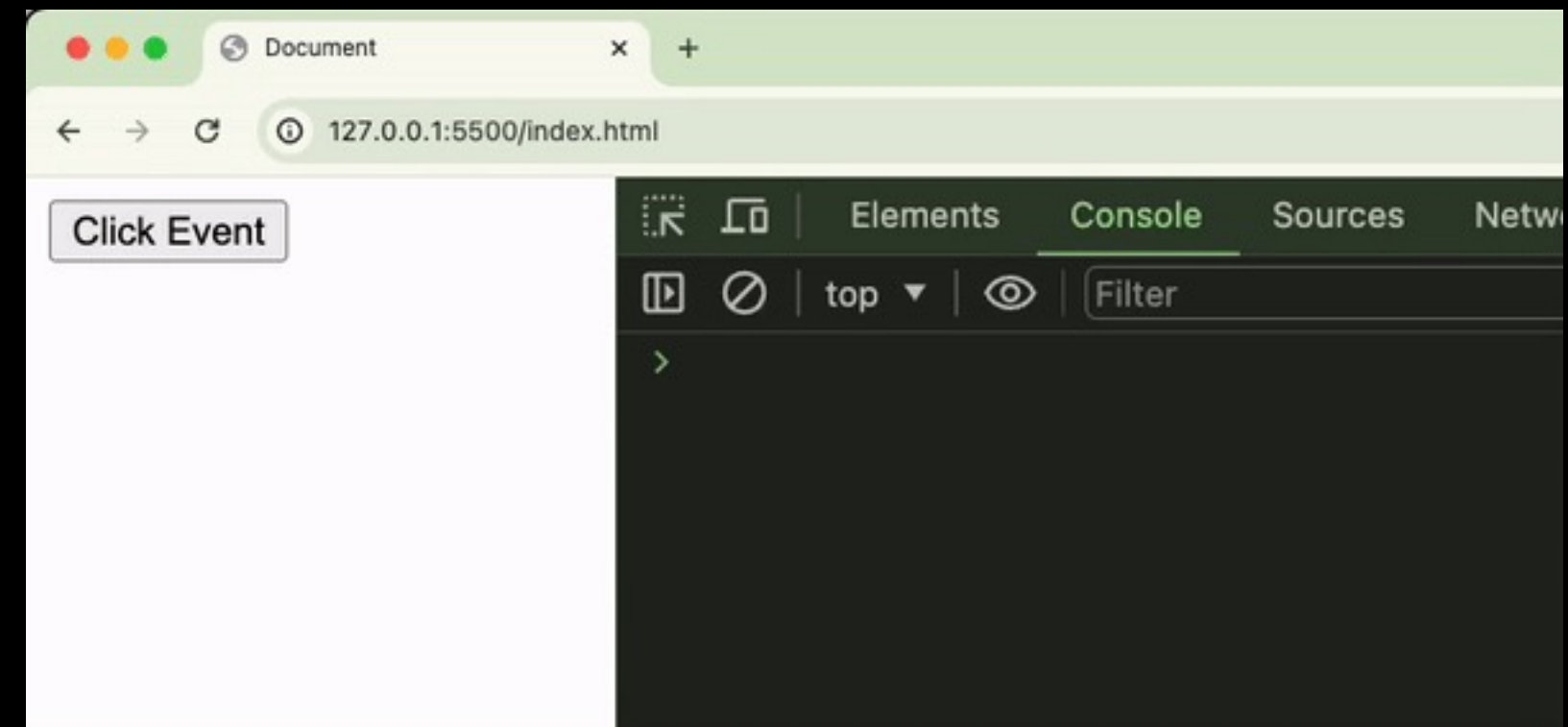
지정된 이벤트가 발생하면, 그 요소에 등록된 이벤트 핸들러를 실행한다.



이벤트 핸들러 등록 - HTML 속성

```
<button onclick="handleClick()">Click Event</button>
...
<script>
  function handleClick() { console.log("button clicked"); }
</script>
```

```
<button onclick="console.log('button clicked')">Click Event</button>
```



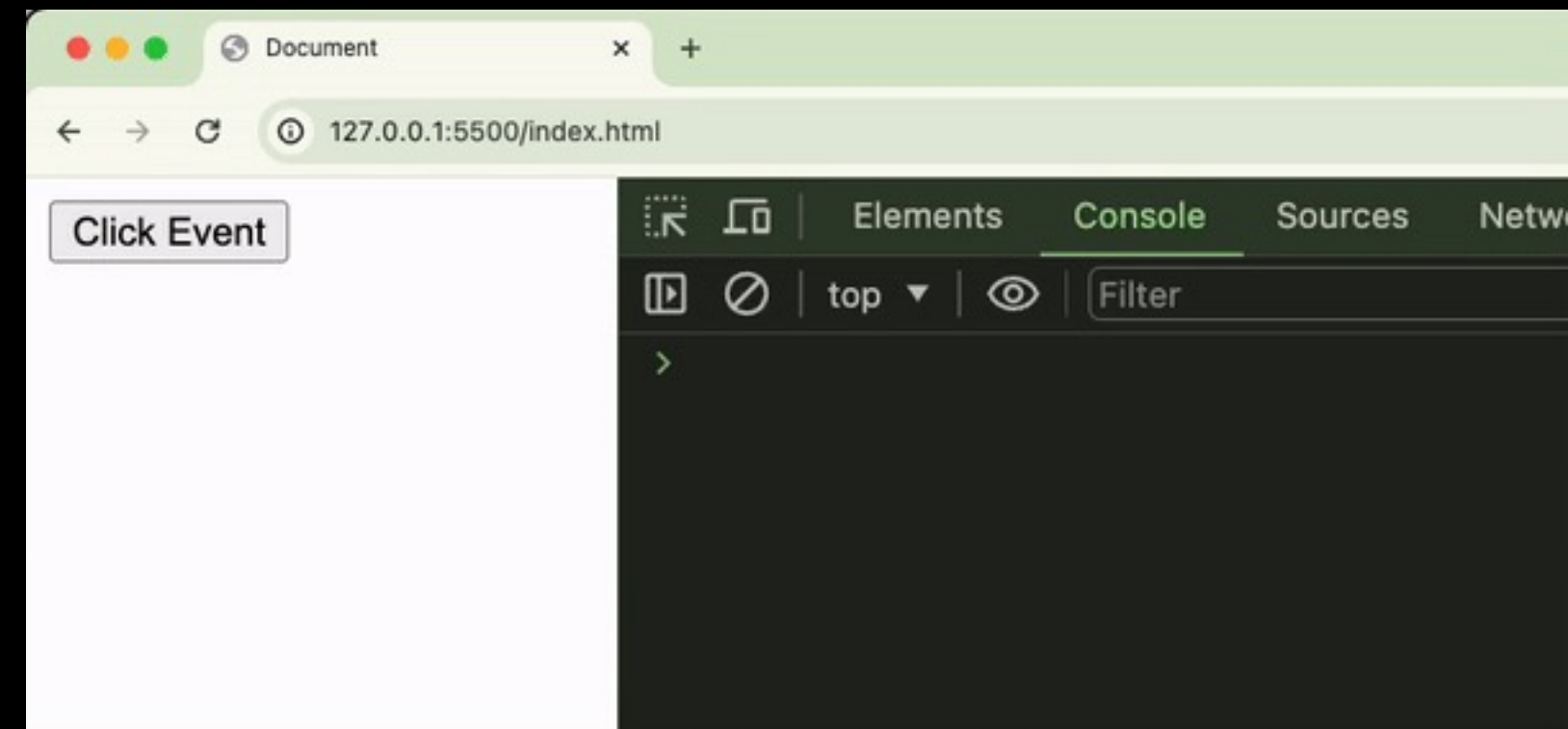
HTML 요소의 `on이벤트명` 속성에 동작 추가하는 방법

HTML 코드에 JS 코드가 포함되는 방식으로, 권장되지 않는다.



이벤트 핸들러 등록 - node 객체 프로퍼티

```
var button = document.getElementById("btn");  
button.onclick = function () { console.log("button clicked"); }
```

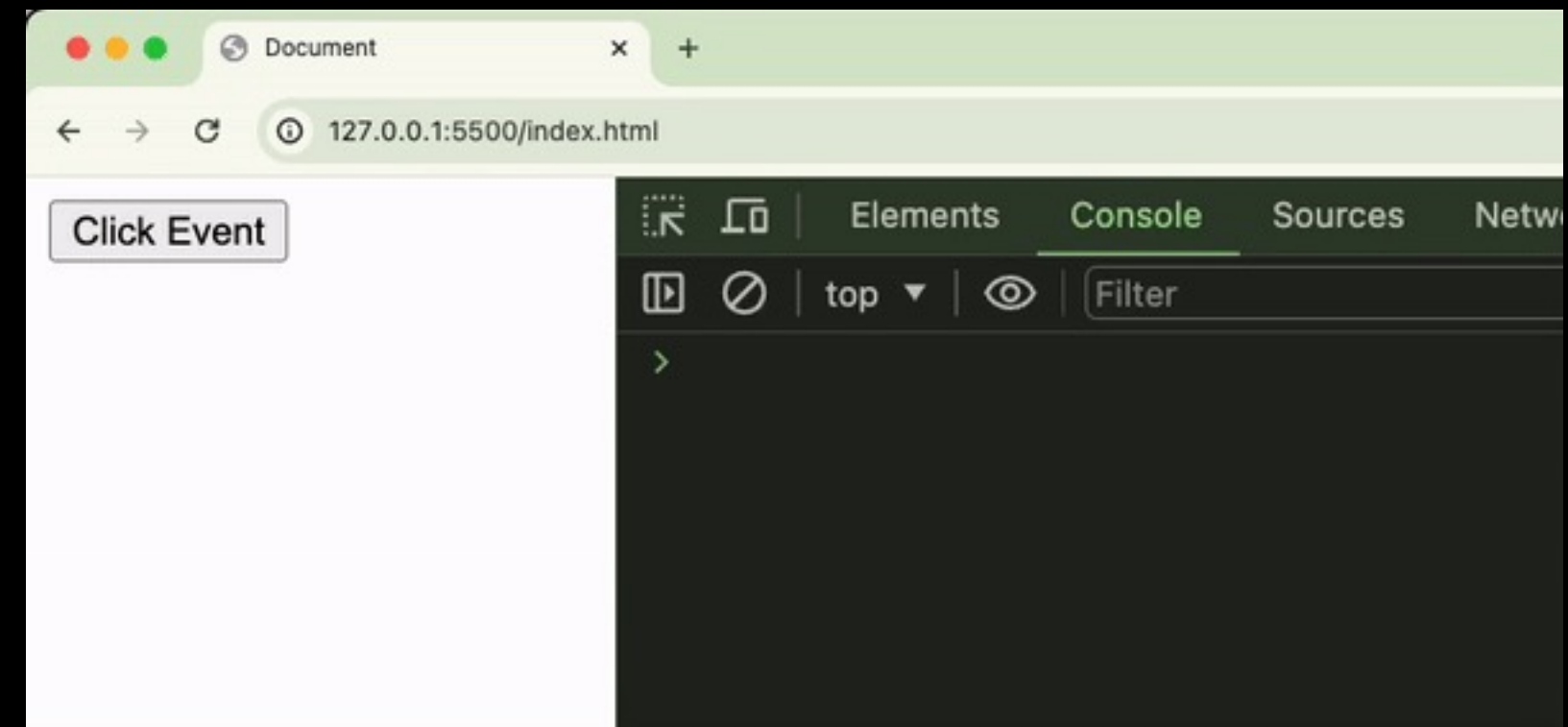


node 객체의 `on이벤트명` 프로퍼티에 핸들러 함수를 등록하는 방법

이벤트 핸들러 등록 - addEventListener

```
var button = document.getElementById("btn");

button.addEventListener("click", function () {
    console.log("button clicked");
});
```



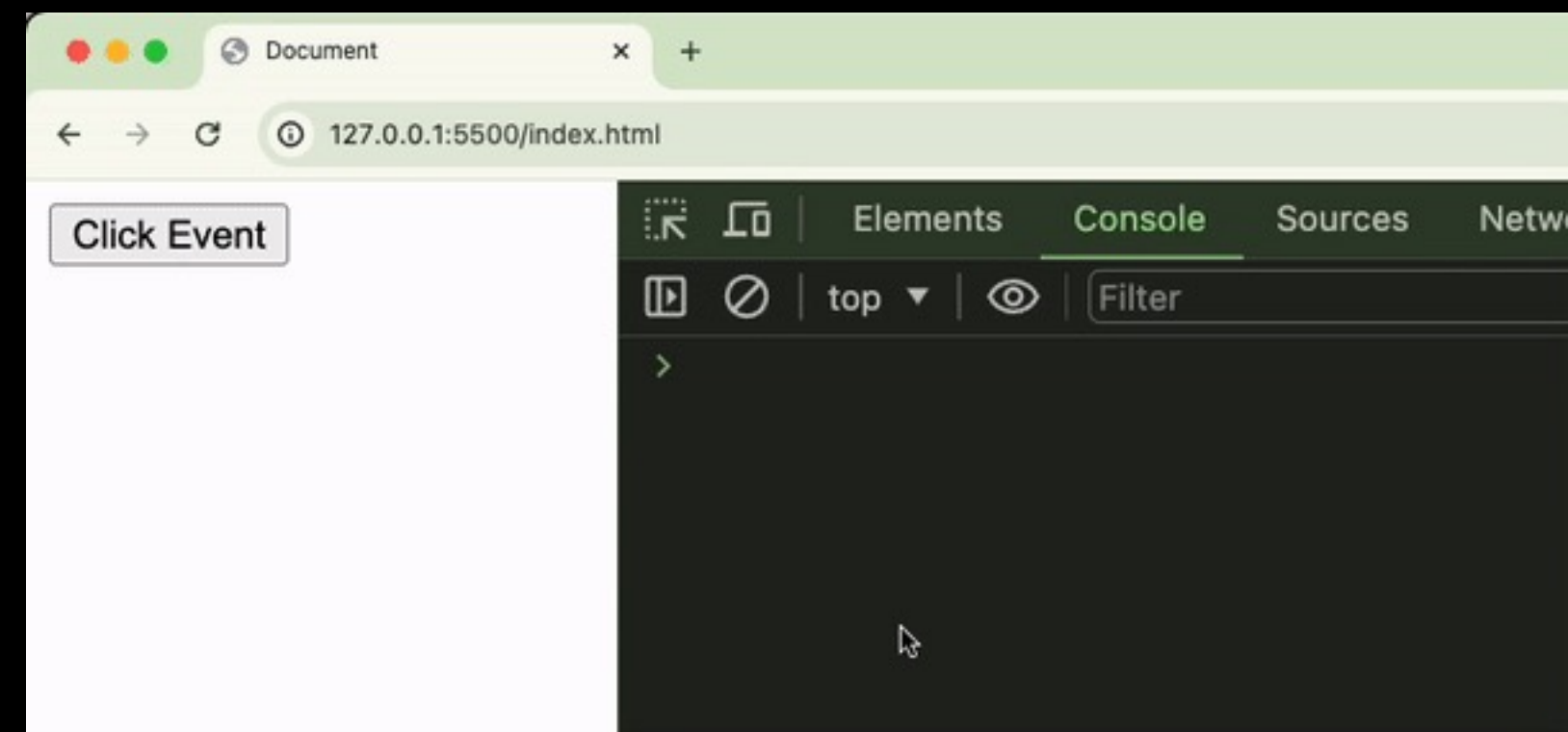
첫 번째 인수로 `이벤트명` 전달, (on을 붙이지 않음)

두 번째 인수로 핸들러 함수 전달



event 객체

```
var button = document.getElementById("btn");  
  
button.addEventListener("click", function (event) {  
    console.log(event);  
});
```



이벤트 핸들러 함수에 첫 번째 인수로 전달되는 객체
→ 해당 이벤트에 대한 정보와 기능이 담겨 있다.



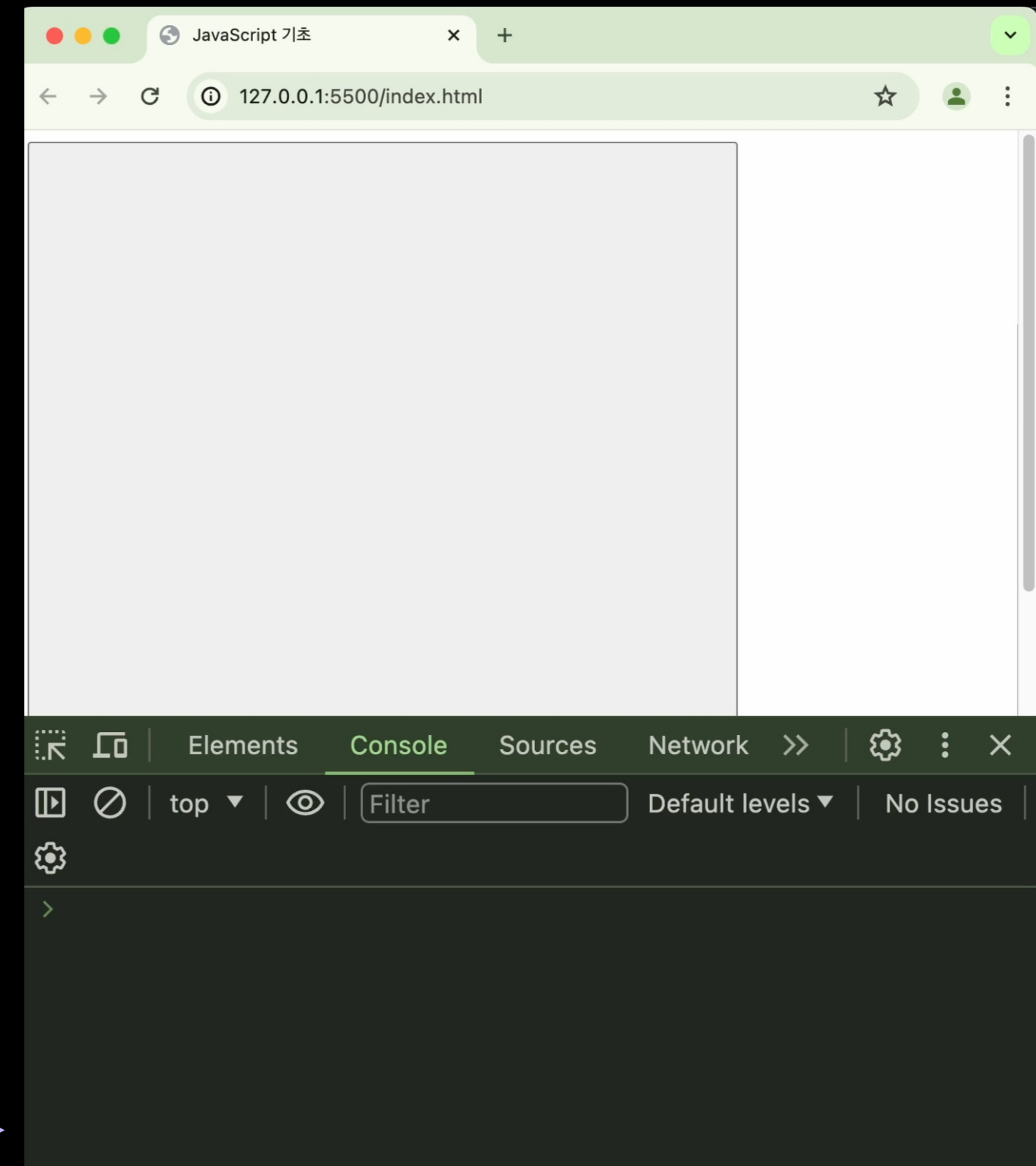
event 객체 - 마우스 정보

```
<button id="big-button" style="width: 500px; height: 500px"></button>

<script>
  var box = document.getElementById("big-button");

  box.addEventListener("click", function (event) {
    console.log(event.clientX, event.clientY);
  });
</script>
```

Play →



마우스 관련 이벤트의 경우 event 객체를 통해
마우스에 대한 정보를 얻을 수 있다. (위치, 클릭 정보 등)



event 객체 - 키보드 정보

```
<h3>
  Pressed: "<span style="font-weight: 900; color: red" id="key"></span>"
</h3>

<script>
  window.addEventListener("keypress", function (event) {
    document.getElementById("key").innerText = event.key.toUpperCase();
  });
</script>
```



↑
Play

키보드 관련 이벤트의 경우 event 객체를 통해
키보드에 대한 정보를 얻을 수 있다. (누른 키 정보 등)

이처럼 **각 이벤트 발생에 대한 정보**를 event 객체를 통해 얻을 수 있다.



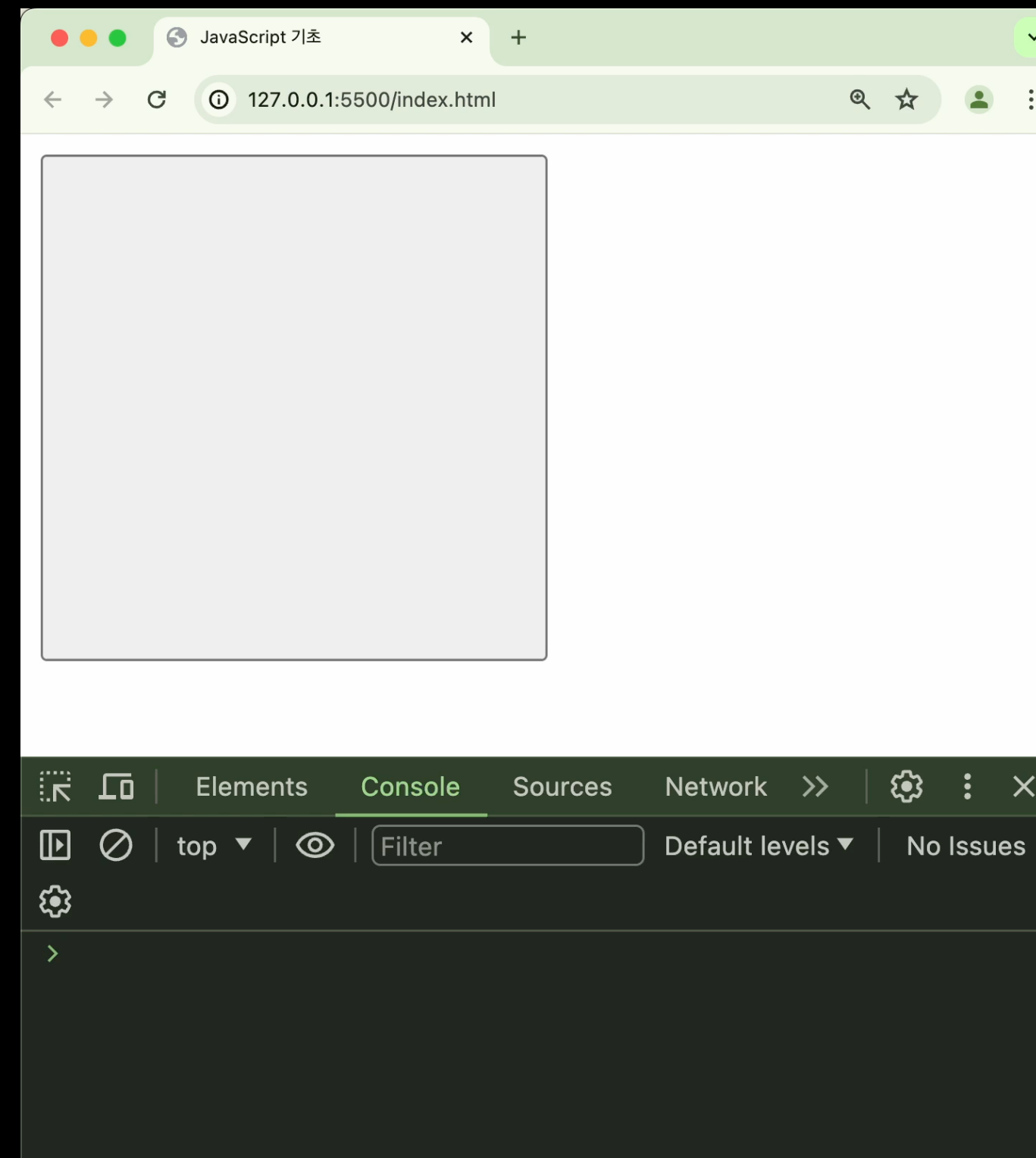
event 객체 – target

```
<button id="big-button" style="width: 200px; height: 200px"></button>

<script>
  var box = document.getElementById("big-button");

  box.addEventListener("click", function (event) {
    console.log(event.target);
  });
</script>
```

Play →



`event.target` 프로퍼티를 통해 이벤트가 발생한 타겟 요소의 node 객체에 접근할 수 있다.



event 객체 – preventDefault

```
<form id="login-form">
  <input type="text" placeholder="아이디" />
  <input type="password" placeholder="비밀번호" />
  <input type="submit" />
</form>
```

A screenshot of a web browser window. The address bar shows '127.0.0.1:5500/index.html'. The page content displays a login form with two input fields: the first is labeled '아이디' (ID) and the second is labeled '비밀번호' (Password). To the right of these fields is a button labeled '제출' (Submit).

form 태그에서 submit 버튼의 **기본 동작**은

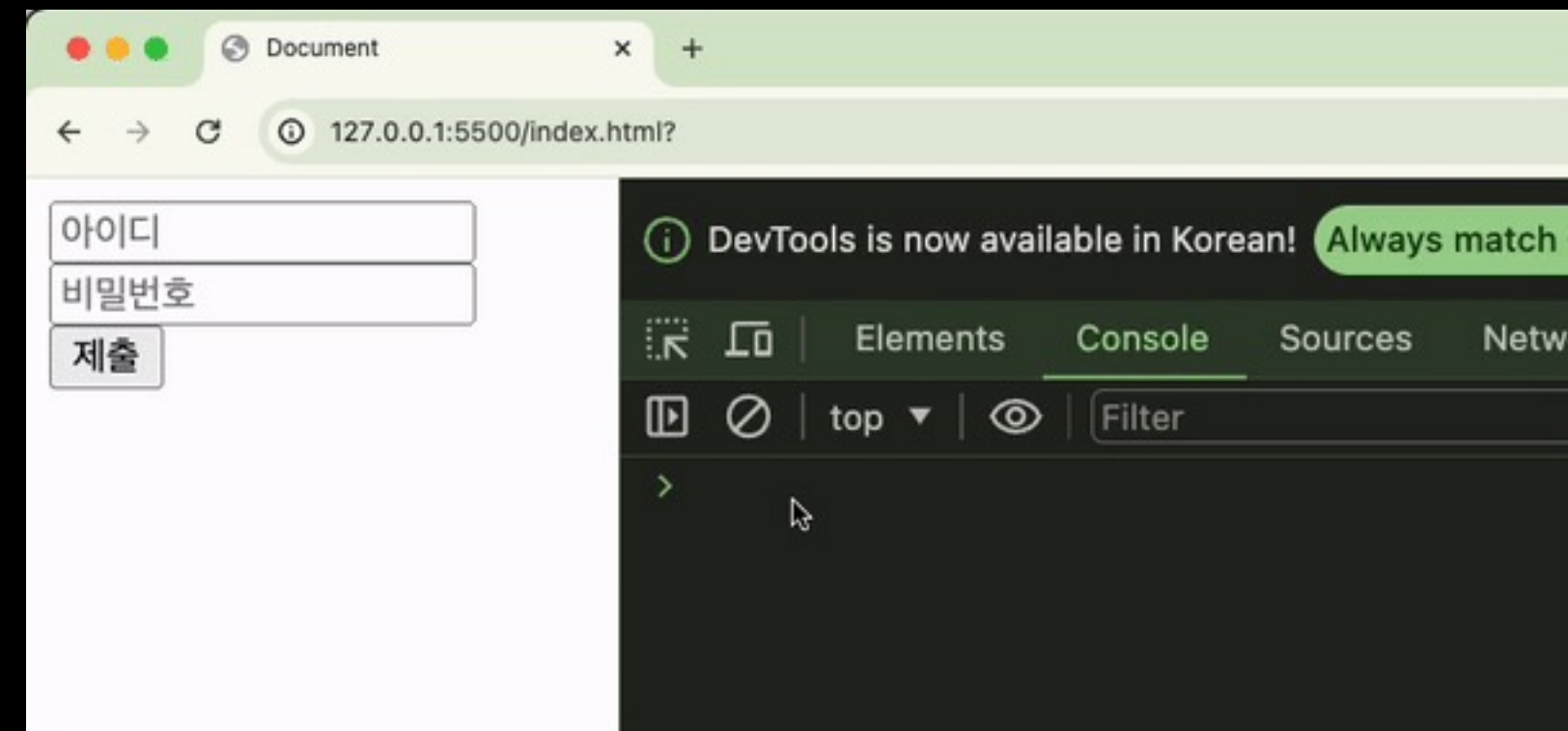
form 제출 → 새로고침



event 객체 – preventDefault

```
var loginForm = document.getElementById("login-form");
var idInput = document.getElementById("id");
var passwordInput = document.getElementById("password");

loginForm.addEventListener("submit", function (event) {
  event.preventDefault();
  console.log("아이디: ", idInput.value);
  console.log("비밀번호: ", passwordInput.value);
});
```



이 기본 동작을 막기 위해 event 객체의
`event.preventDefault()` 메서드 사용한다.

이벤트 핸들러 제거 – removeEventListener

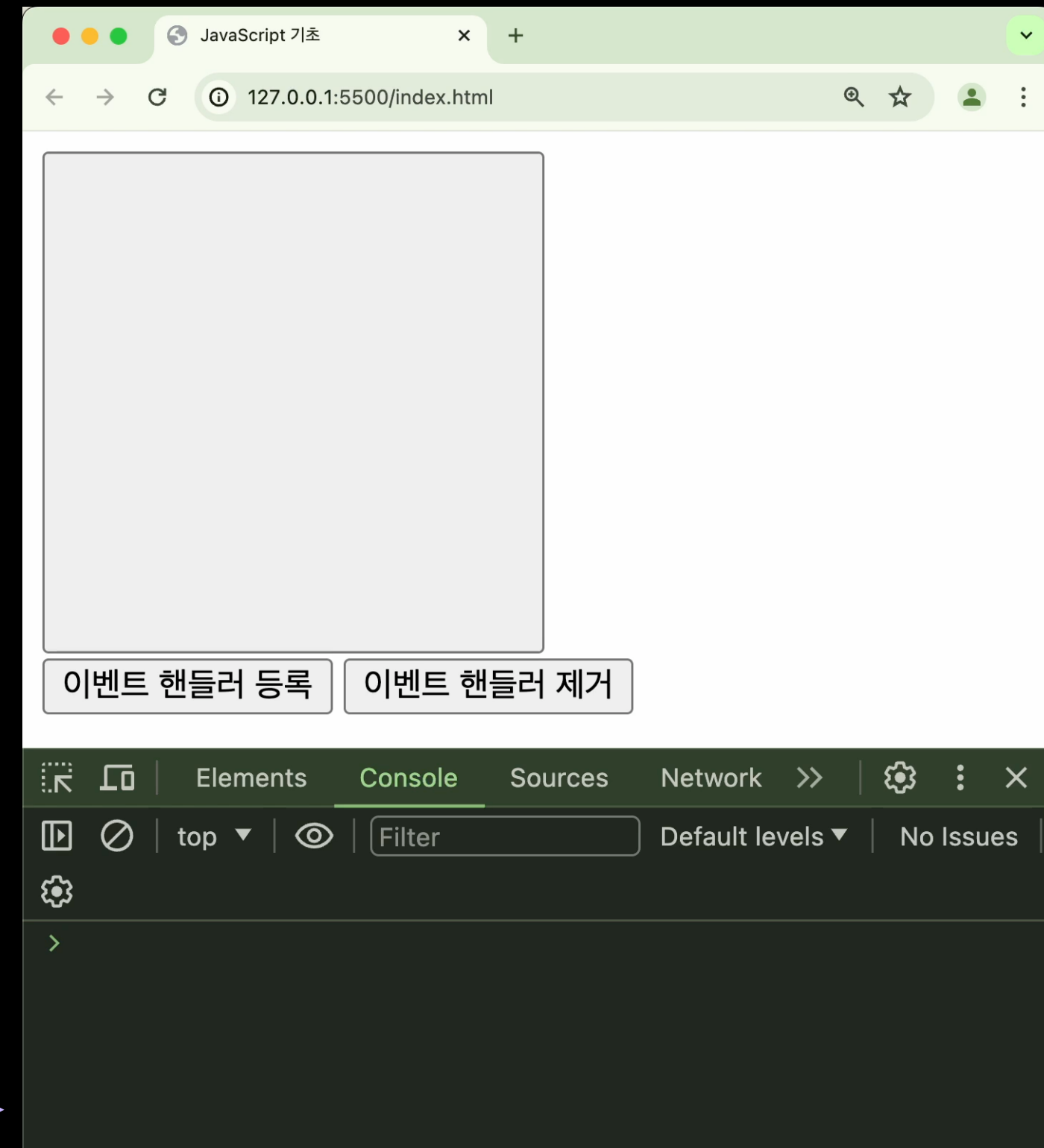
```
<button id="big-button" style="width: 200px; height: 200px"></button>
<br />
<button id="register">이벤트 핸들러 등록</button>
<button id="remove">이벤트 핸들러 제거</button>
<script>
  var box = document.getElementById("big-button");
  var registerBtn = document.getElementById("register");
  var removeBtn = document.getElementById("remove");

  function handleBtnClick(event) {
    console.log(event.clientX, event.clientY);
  }

  registerBtn.addEventListener("click", function () {
    box.addEventListener("click", handleBtnClick);
  });

  removeBtn.addEventListener("click", function () {
    box.removeEventListener("click", handleBtnClick);
  });
</script>
```

Play →



‘removeEventListener’ 메서드로 등록된 이벤트 핸들러를 제거할 수 있다.